p-value estimation

In this notebook I test empyrically my theoretical function for the estimation of the p-value, given the observed frequency of successful tests. In particular, when 0 successes over n tests are observed, my formula provides an expected value for the p-value of

$E[p] = \frac{1}{n+2}$

differently from the one proposed in *Phipson et al. (2010)*, which provides a value of

$\frac{1}{n+1}$

Uniform prior

We assume a uniform prior for the value of the "true p-value". Any number in $[0, 1]$ is equally likely.

In [1]:
```
import random
import pandas as pd
import matplotlib.pyplot as plt

def choose_a_true_p(lower=0, upper=1):
    ''' Returns a random value of the "true p-value" to be used. '''
    return random.uniform(lower, upper)
```

Samples are random

We can generate a sample of size *n*, where successes are produced with a probability equal to a chosen p-value, using the following function. Each element in the sample has a probability equal to *true_p* to be a success.

In [2]:
```
def get_sample(n, true_p):
    ''' Returns a sample of size n, where each element has a probability true_p
    of being True, and a probability 1-true_p of being False.
    true_p stands for "true p-value". '''
    sample = []
    for i in range(n):
        if random.random() < true_p:
            sample.append(True)
        else:
            sample.append(False)
    return sample
```

We are interested in knowing when a sample doesn't cointain any successes. indeed, we are going to collect examples of sets of size n with 0 successes.

In [3]:
```
def is_0_frequency(sample):
    ''' Returns True if all the elements in the sample are False.
    Returns False otherwise. '''
    return not sum(sample)
```

Estimating the average p-value

My formula estimates the expected value of the *p-value*, given a uniform prior, and given that 0 out n tests were successes.

The following function provides an approximation by performing many "experiments". Each experiment consists of choosing randomly a value for the "true p-value", and then generating a sample of size n, based on that p-value. Whenever a p-value leads to a sample of $0/n$ successes, its value is recorded. The function returns the average value of the recorded p-values, which corresponds to the expected value of a p-value known to have generated $0/n$ successes.

In [4]:
```
def empyrical_average_true_p(n, n_experiments):
    ''' Performs as many experiments as specified by n_experiments.
    Each experiment consists of choosing randomly a value for the
    "true p-value", and then generating a sample of size n, based on that
    p-value. Whenever a p-value leads to a sample of 0/n successes, its value
    is recorded. The function returns the average value of the recorded
    p-values, which corresponds to the expected value of a p-value known to
    have generated 0/n successes. '''
    true_p_list = []
    for i in range(n_experiments):
        true_p = choose_a_true_p()
        a_sample = get_sample(n, true_p)
        if is_0_frequency(a_sample):
            true_p_list.append(true_p)
    return sum(true_p_list)/len(true_p_list)
```

Test

Here I run a test, chosing $n = 10$.

The computational approximation provided by the *empyrical_average_true_p* function depends on the number of experiments performed: the *n_experiments* parameter. We will study the convergence of the approximation for increasing values of *n_experiments*. # The quality of the approximation will depend on the number of experiments performed. The less accurate estimation will be obtained by performing $10^{SMALLEST\_EXP}$ experiments, while the most accurate will be obtained by perfomring $10^{LARGEST\_EXP}$ experiments.

In [5]:
```
n = 10

SMALLEST_EXP = 3
LARGEST_EXP = 8
```

The two p-value estimators to be compared are the following:

My formula: $\frac{1}{n+2}$

Phipson et al. (2010): $\frac{1}{n+1}$

In [6]:
```
theo_elia = 1/(n+2)
theo_phipson = 1/(n+1)
```

We can run all the study and collect results in a table format, to make the comparison easy.

In [7]:
```
n_experiments_list = []
empyrical_list = []
error_elia_list = []
error_phip_list = []

for exponent in range(SMALLEST_EXP, LARGEST_EXP+1):
    n_experiments = 10**exponent
    print("Estimating E[p-value] with 10^{} experiments ...".format(exponent))

    emp = empyrical_average_true_p(n, n_experiments)
    err_elia = abs(emp - theo_elia)
    err_phip = abs(emp - theo_phipson)

    n_experiments_list.append(n_experiments)
    empyrical_list.append(emp)
    error_elia_list.append(err_elia)
    error_phip_list.append(err_phip)

len_table = len(range(SMALLEST_EXP, LARGEST_EXP+1))
res_table = pd.DataFrame({"n": [n] * len_table,
                          "n_experiments": n_experiments_list,
                          "empyrical": empyrical_list,
                          "1/(n+2)": [theo_elia] * len_table,
                          "1/(n+1)": [theo_phipson] * len_table,
                          "error_elia": error_elia_list,
                          "error_phip": error_phip_list})

print('\nRESULTS TABLE\n------------')
print(res_table)
```

```
Estimating E[p-value] with 10^3 experiments ...
Estimating E[p-value] with 10^4 experiments ...
Estimating E[p-value] with 10^5 experiments ...
Estimating E[p-value] with 10^6 experiments ...
Estimating E[p-value] with 10^7 experiments ...
Estimating E[p-value] with 10^8 experiments ...

RESULTS TABLE
------------
    n  n_experiments  empyrical   1/(n+2)   1/(n+1)  error_elia  error_phip
0  10           1000   0.079364  0.083333  0.090909    0.003969    0.011545
1  10          10000   0.086007  0.083333  0.090909    0.002674    0.004902
2  10         100000   0.084159  0.083333  0.090909    0.000826    0.006750
3  10        1000000   0.083552  0.083333  0.090909    0.000219    0.007357
4  10       10000000   0.083265  0.083333  0.090909    0.000069    0.007645
5  10      100000000   0.083305  0.083333  0.090909    0.000028    0.007604
```

The table shows that the computational approximation of $E[p]$ approaches $\frac{1}{n+2}$. Indeed, using my formula the error approaches 0, while using Phipson's formula the error approaches the difference between my formula and their formula, $\frac{1}{n+1} - \frac{1}{n+2}$, which is

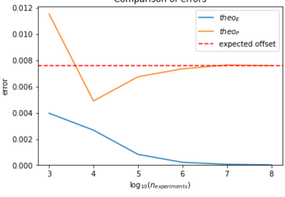
In [8]:
```
expected_offset = theo_phipson - theo_elia
print(expected_offset)
```

```
0.007575757575757583
```

(this is the error you would expect when using Phipson's formula if $E[p]$ is not $\frac{1}{n+1}$, but $\frac{1}{n+2}$)

We can also see the convergence using line plots.

In [9]:
```
x = list(range(SMALLEST_EXP, LARGEST_EXP+1))
plt.title('Comparison of errors')
plt.xticks(x)
plt.xlabel(r'$\log_{10}(n_{experiments})$')
plt.ylabel('error')
plt.plot(x, res_table['error_elia'], label='$theo_E$')
plt.plot(x, res_table['error_phip'], label='$theo_P$')
plt.axhline(y=(theo_phipson - theo_elia),
            c='r', linestyle='dashed', label='expected offset')
plt.ylim(bottom=0)
plt.legend()
plt.show()
```



I'm going to assume that this doesn't mean that they made a mistake. For some reason, they argue that $\frac{1}{n+1}$ is an ideal estimator in some context. To me, it is equal to $p(m = 0)$ in my formula, so it doesn't really makes sense as an estimator of the p-value (we are not interested in the probability of seeing 0 successes, but rather in the value of the p-value *given* that we have observed 0 successes. But other things can be numerically equal to $\frac{1}{n+1}$, which could be an ideal estimator of the p-value. I guess I'll have to take the time to read their paper and see what is precisely that they are saying.